

문제 1 · Autonomous Driver

한강 무인 수상정 자율운항

무인 수상정이 출발 선착장을 떠나 지정된 경유점(waypoint)을 순서대로 통과한 뒤 목적지에 도착해야 합니다. 운항 중 부표, 교각, 정박 선박, 이동 중인 선박과 충돌해서는 안 되며, 과도한 우회와 지연은 감점 요인입니다. 참가자는 동일한 운항 과제를 4가지 관측 조건에서 수행하는 자율운항 에이전트를 개발해 제출합니다.

제출물

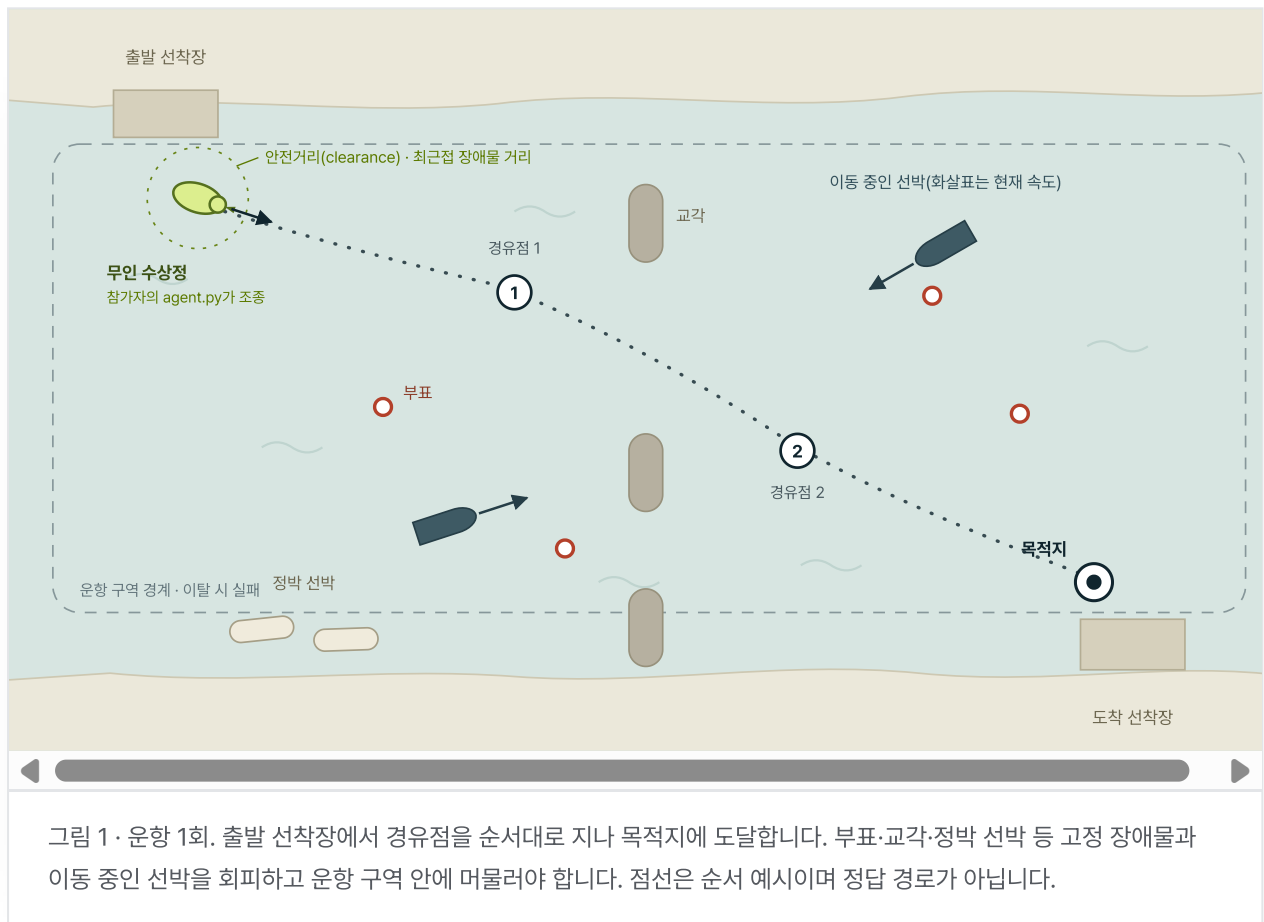
매 제어 주기(control tick)마다 관측을 입력받아 목표 전진속도와 회전속도를 출력하는 `agent.py` 1개. 관측 조건별 인식(perception) 모듈은 달라도 되나, 최종 제어 판단은 하나의 제출물 안에서 이루어져야 합니다.

THE SCENARIO

문제 개요: 무인 수상정 자율운항

문제 상황은 한강의 여러 선착장 사이를 오가는 무인 수상정의 시험 운항입니다. 운항마다 출발 지점·목적지·중간 경유점을 지정합니다. 운항 구역에는 강변·선착장 등의 경계, 부표와 구조물, 강 가운데의 암반, 정박 선박, 그리고 선착장 사이를 정해진 차선으로 오가는 다른 선박이 배치됩니다. 경유점은 강폭을 가로질러 놓이며, 하류로 내려간 뒤 되돌아오는 왕복 코스도 있습니다.

주최 측은 이 상황을 재현한 공개·비공개 시뮬레이터에서 평가합니다. 참가자의 프로그램은 고정된 제어 주기마다 관측을 입력받아 다음 속도 명령을 반환합니다. 시뮬레이터는 명령과 선체 반응을 적용해 다음 관측을 전달하며, 이 순환은 목적지 도착·충돌·시간 초과까지 반복됩니다.



운항 1회의 절차

route → observe → act → move

01 · ROUTE

출발점·경유점·목적지와 운항 구역 설정

02 · OBSERVE

해당 조건의 센서 관측과 공통 운항 정보 수신

03 · ACT

다음 전진속도와 회전속도 명령 반환

04 · REPEAT

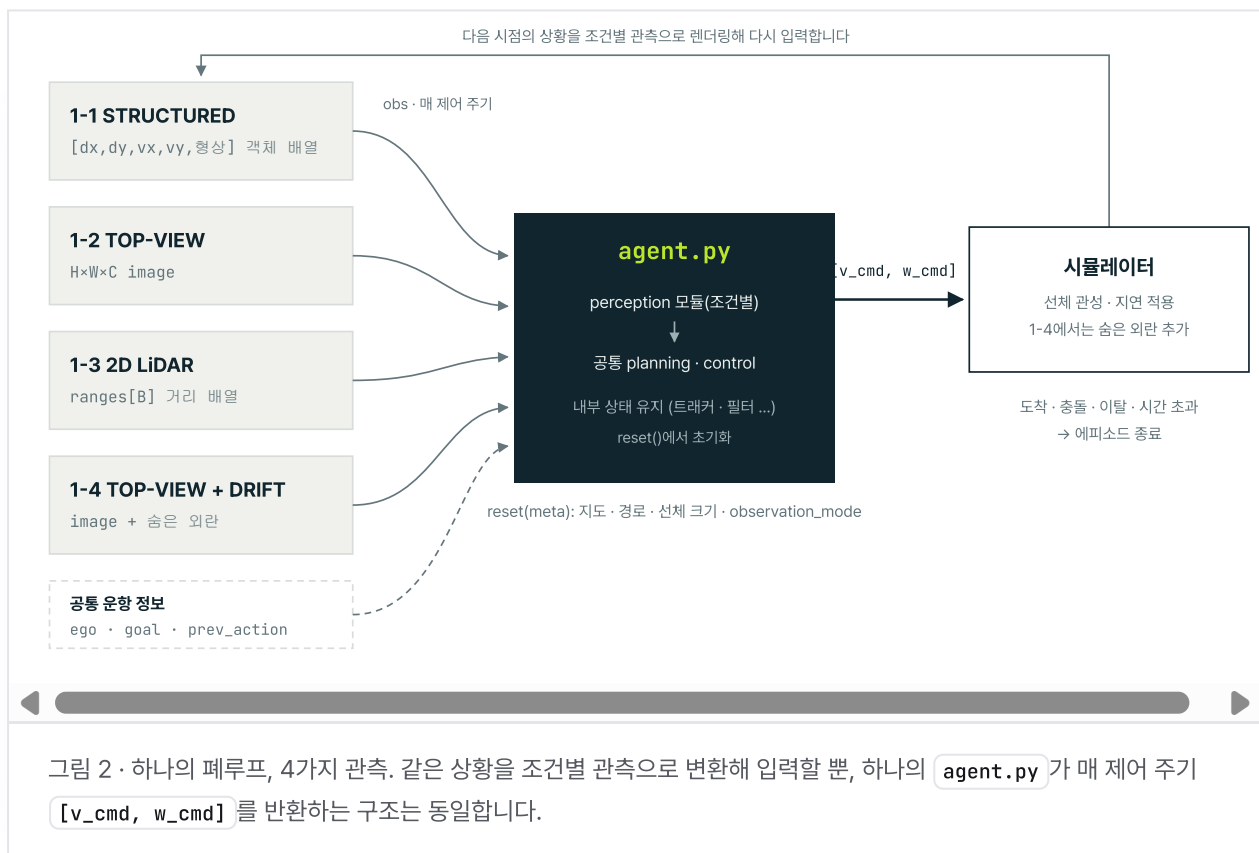
도착·충돌·이탈·시간 초과까지 페루프(closed-loop) 실행

공통 과제

항행 구역 경계와 경유점은 에피소드 시작 시 제공되고, 장애물과 교통 배치는 공개되지 않습니다. 주어진 관측만으로 수상정을 충돌 없이 목적지까지 운항

관측 조건 간 공통 에피소드 사용

4가지 관측 조건은 동일한 지도·교통 에피소드 집합을 공유합니다. 조건마다 지도나 장애물 구성을 달리한 별개의 문제가 아니라, 동일한 상황을 서로 다른 관측 정보로 얼마나 안정적으로 해결하는지를 비교하는 구성입니다.



결과물과 제공 자료

결과물은 센서 관측으로부터 다음 제어 명령을 계산하는 프로그램입니다. 구현 방법은 제한하지 않으며, 특정 신경망이나 주최 측 베이스라인의 학습을 요구하지 않습니다. 베이스라인은 환경 실행·제출 인터페이스·채점 방법을 보여주는 최소 예제입니다.

제공 자료	용도
공개 시뮬레이터	수상정 반복 운항과 궤적·실패 사례 수집
공개 paired 스테이지	동일한 지도와 교통을 서로 다른 관측 조건에서 비교
<code>baseline_agent.py</code>	실행 가능한 최소 예제. 수정하거나 새로 작성해도 됨
<code>agent_template.py</code>	<code>Agent</code> 함수, 관측 스키마, 액션 형식 확인
<code>self_score.py</code>	조건별 완주 여부와 <code>P·C·T</code> 원시값 확인. 상대점수는 마감 후 전체 참가자 기준으로 산출
<code>RULES.md</code>	실행 환경, 자원 제한, 외부 모델 사용 규정과 금지 사항 확인

학습 여부	공개 시뮬레이터로 데이터를 생성해 학습해도 되고, 학습 없이 알고리즘만 작성해도 됩니다. 평가 대상은 방법이 아니라 비공개 에피소드의 운항 결과입니다.
-------	--

공통 입출력 규격

하나의 `Agent` 가 4가지 관측 조건 전체를 실행합니다. 에피소드 시작 시 `reset()` 으로 운항 구역과 관측 조건을 받고, 이후 `act()` 가 반복 호출됩니다.

```
agent = Agent(config_path)
agent.reset(meta)          # 지도·경로·선체 크기·observation_mode
action = agent.act(obs)    # 매 control tick마다 [v_cmd, w_cmd] 반환
```

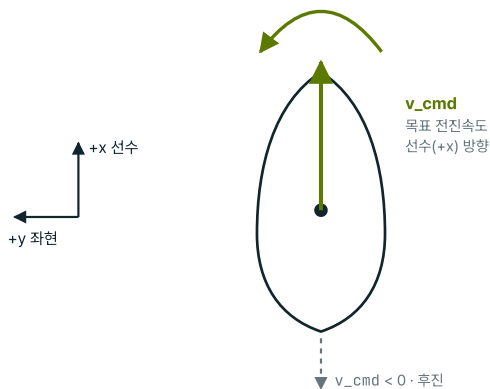
`v_cmd`

throttle 출력이 아닌 수상정의 **목표 전진속도**입니다. 시뮬레이터의 자동조종기가 이 목표값을 추진기 명령으로 바꿔 수상정(Track 2 참가자 키트에 공개된 명목 선박 모델과 같은 선체)을 움직이므로, 관성 때문에 실제 속도는 즉시 바뀌지 않고, 선회하면 선미가 옆으로 밀리며 미끄러지고, 전속에서 급선회하면 속도가 떨어집니다. 목표 속도를 낮추면 감속은 추진을 줄여 관성으로 흘러가는 방식이며, 음수 명령만 후진합니다.

`w_cmd`

rudder 각도가 아닌 수상정의 **목표 yaw rate**입니다. 부호는 좌·우 회전 방향, 단위는 rad/s입니다.

$w_cmd > 0$: 좌선회 (위에서 볼 때 반시계)



명령과 실제 응답

v_cmd · w_cmd 모두 같은 방식

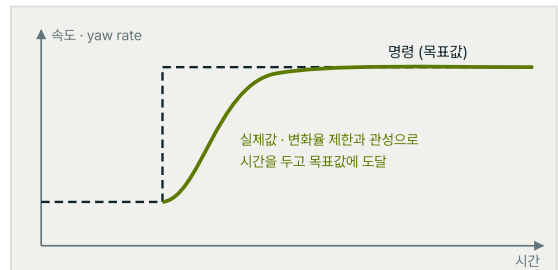


그림 3 · 조종 명령이 선체에 작용하는 방식. `v_cmd` 는 선수(+x) 방향의 목표 전진속도이며 음수면 후진합니다. `w_cmd` 는 목표 yaw rate로, 양수면 위에서 볼 때 반시계, 즉 좌현으로 돌고 음수면 우현으로 둡니다. 두 값을 함께 주면 반경 약

v/w의 호를 그리며 선회합니다. 명령은 요구값이며 시뮬레이터가 변화율 제한과 관성을 적용하므로, 실제 속도와 yaw rate는 오른쪽 그래프처럼 시간을 두고 목표값에 도달합니다.

전 조건 공통 입력 필드

필드	의미
t	reset() 이후의 제어 틱 번호
pose	월드 좌표 기준 위치와 heading. 항행 구역 경계가 월드 좌표로 공개되므로 자기 위치를 함께 제공합니다
vel	대지속도의 선체축 성분과 yaw rate. 횡방향 성분은 선회할 때 선체가 미끄러지는 양이며, 1-4에서는 여기에 외란에 밀린 양이 더해집니다. 자기 선회로 설명되지 않는 부분이 외란 추정의 단서입니다
wp_index	현재 목표 경유점의 순번. 통과 판정은 시뮬레이터가 하므로 에이전트가 기억할 필요는 없습니다
wp_polar	현재 목표까지의 거리와 수상정 정면 기준 상대 방위
prev_action	직전 틱에 실제로 적용된 명령. 변화율 제한이 걸리거나 시간 초과로 이전 명령이 재사용되면 요청값과 다를 수 있으므로 적용값을 제공합니다
perception	조건별 상이 — 구조화 객체 배열, top-view 래스터 또는 2D LiDAR 스캔

각도는 모두 라디안이며 $(-\pi, \pi]$ 범위로 감아서 제공합니다. 선체축은 +x가 선수, +y가 좌현이고, yaw rate와 w_cmd의 양수는 좌선회입니다. 필드 순서와 dtype은 스타터 킷의 스키마 문서에 있습니다.

내부 상태

act()가 매년 독립적으로 판단할 필요는 없습니다. 이전 관측, 출력한 액션, 트래커·필터·순환 상태를 에피소드 동안 유지하고 reset()에서 초기화합니다.

수치 규격은 스타터 킷에

이 문서는 각 필드가 무엇인지와 왜 그렇게 주어지지만 설명합니다. 필드 순서와 dtype, 래스터 해상도와 픽셀 축척, LiDAR 빔 수와 사거리, v_cmd/w_cmd의 수치 범위와 제어 주기, 장애물 크기 범위, 외란 세기는

모두 확정되었으며 스타터 킷의 스키마 문서와 예시 관측 덤프에 한 곳으로 모아 둡니다. 값이 한 문서에만 있으면 설명서와 규격이 서로 어긋날 수 없습니다. **실행 시간 제한만** 채점 환경에서 실측해 확정하며, 시간은 점수에 반영되지 않는 안전장치이므로 넉넉하게 잡습니다. 참고로 **채점은 실시간으로 돌지 않습니다** — 제어 주기는 시뮬레이션 시간이고 시뮬레이터가 `act()` 를 기다리므로, 제어 주기를 벽시계 마감으로 읽고 모델 크기를 그에 맞추는 필요가 없습니다.

4가지 관측 조건

4가지 조건은 같은 종류의 지도·목표·정적·동적 장애물·기본 수상정 동역학을 공유합니다. 1-1부터 1-3까지는 perception 표현이 다르고, 1-4는 1-2와 같은 입력에 숨은 외란만 추가합니다.

1-1

장애물 위치와 속도의 수치 입력

주변의 고정·이동 장애물을 객체 단위로 제공합니다. 각 행은 수상정 기준 위치·대지속도·충돌 형상 파라미터를 담으며 고정 장애물의 속도는 0입니다. 속도와 위치 모두 선체축 성분으로 주므로 한 행에 두 좌표계가 섞이지 않습니다. 가려진 물체도 목록에 포함되고, 배열은 고정 길이에 유효성 마스크가 붙습니다. 별도의 객체 인식 없이 충돌 예측과 경로 결정이 가능합니다.

입력

[dx, dy, vx, vy, 형상] 고정 길이 배열 + 유효 마스크

남는 문제

Prediction · planning · control

역할

STRUCTURED

perception 오차가 없는 성능 상한

1-2

운항 구역의 top-view 영상 입력

수상정과 주변 구조물·선박이 표시된 top-view 래스터를 제공합니다. 스튜디오 지도와 같은 코드로 그린 같은 그림이며, 채점 래스터에는 노이즈가 없습니다. 노이즈는 스튜디오에서 참가자가 직접 넣어 시험합니다. 래스터는 선체 기준으로 그려지므로 선수가 항상 위를 향합니다(**bow_up**). 객체 속도나 ID는 제공하지 않으므로 필요하면 연속 프레임을 비교해 객체를 탐지·추적해야 합니다. 영상을 정책에 직접 입력하거나 detection·segmentation·tracking 결과로 변환해 사용할 수 있습니다.

입력

top-view 래스터 · 100 m 정사각 · **bow_up**

추가 문제

Visual perception · tracking

TOP
VIEW

허용

자유로운 전처리와 학습 모델

1-3

주변 표면까지의 거리 입력

수상정 주위로 발사한 빔별 첫 번째 반사(return) 거리를 제공합니다. 반사 대상의 종류(선착장·부표·다른 선박)는 표시하지 않으며, 가려진 물체와 속도도 관측되지 않습니다. 현재 스캔과 운항 이력을 결합해 주변 구조와 움직임을 복원해야 합니다.

입력

선수 0번에서 반시계 순서의 `ranges` 배열 · 360°

추가 문제

Occlusion · temporal fusion

NO RETURN

최대 사거리 값으로 표기 — `0` 이 아닙니다

2D
LiDAR

1-4

top-view 영상 입력과 숨은 외란

입력과 기본 스테이지는 1-2와 같습니다. 다만 조류나 바람에 해당하는 저주파 외력이 에피소드 내내 수상정의 운동을 밀어냅니다. 외란의 현재 값은 관측에 포함되지 않으므로 명령과 실제 이동의 차이를 이력에서 추정해 제어를 보정해야 합니다.

직접 제공하지 않음

외란 방향·세기·현재 값

필요한 능력

Online estimation · adaptive control

대조 조건

1-2와 paired 스테이지 사용

TOP
VIEW +
DRIFT

외란 정보의 공개 범위

실제 수상정에는 현재 조류를 알려 주는 센서가 없습니다. 같은 속도 명령에 예상보다 옆으로 밀리거나 회전이 늦어지는 변화를 누적해 외란을 추정해야 합니다. 1-4는 이 온라인 추정 능력을 평가합니다.

공개 항목	비공개 항목
외란이 연속적이고 완만하게 변한다는 사실	매 스텝의 실제 외란 벡터
외란 유형(family), 최대 세기, 대략적인 변화 시간 척도	비공개 에피소드의 시드·방향·시간 변화
동일한 외란 유형을 적용한 공개 연습 스테이지	사전 계산된 보정값이나 특권 상태(privileged state)

- ✓ 비공개 평가에는 공개 환경에 없던 외란 유형을 추가하지 않습니다.
- ✓ 외란은 관측 이력으로 추정 가능한 완만한 과정이며, 순간적으로 방향이 뒤집히는 적대적 외력은 사용하지 않습니다.

평가 기준: 완주와 상대점수

충돌·운항 구역 이탈이 발생한 에피소드는 0점이고, 시간 초과는 도달한 경유점에 따라 부분점수를 받습니다(아래). 완주한 에피소드에서는 제어 세기·안전거리·시간 효율 3가지를 기록합니다. 각 항목은 **같은 에피소드를 완주한 참가자들의 최고 기록**과 비교해 상대점수로 환산하며, 그 에피소드의 최고 기록 참가자가 1을 받습니다.

제어 세기 F 가중치 0.3	안전거리 C 가중치 0.4	시간 효율 T 가중치 0.3
--------------------	-------------------	--------------------

완주하지 못한 에피소드는 그 에피소드의 최고 기록 산정에 들어가지 않습니다. 아무것에도 닿지 않은 채 시간 한도에 도달한 에피소드(시간 초과)는 **(순서대로 도달한 경유점 수 ÷ 전체 경유점 수)² × 그 에피소드 완주 참가자 중 가장 낮은 점수**를 받으며, 자신의 제어 세기·안전거리·시간 기록은 쓰지 않습니다. 따라서 시간 초과는 어떤 완주보다도 낮고, 목적지에 가까이 갈수록 많이 받습니다. 충돌·운항 구역 이탈·제출물 오류는 0점입니다.

집계는 **에피소드 단위로 먼저 정규화**합니다. 한 에피소드에서 어떤 항목의 상대점수는 그 에피소드를 완주한 참가자들의 기록 중 최고와 비교한 비율입니다. 조건 점수는 이렇게 얻은 에피소드별 비율을 해당 조건의 비공개 에피소드 전체에 대해 평균한 값입니다($\hat{F}$, $\hat{C}$, $\hat{T}$). 기준값을 따로 계산하지 않습니다 — 같은 코스를 달린 참가자끼리만 비교하기 때문입니다.

$$\text{조건 점수} = 100 \times (0.3 \hat{F} + 0.4 \hat{C} + 0.3 \hat{T})$$

어느 참가자도 완주하지 못한 에피소드는 모든 참가자에게 0점이며, 시간 초과와 부분점수도 없습니다. 최고 기록은 제출 마감 후 에피소드·항목마다 1회 계산합니다. 대회 기간에는 리더보드가 그때까지 제출된 참가자들을 기준으로 같은 방식으로 계산해 주므로, 주최 측이 기준값을 따로 공개하지 않아도 자기 위치를 확인할 수 있습니다. 리더보드는 **자기 제출물의 에피소드별 점수**를 함께 보여주므로 어느 에피소드에서 막히는지도 알 수 있습니다. 리더보드와 최종 점수는 같은 비공개 에피소드 묶음을 사용합니다. 에피소드별 항목 정의는 다음과 같습니다.

항목	계산 방법
S · 완주	경유점을 순서대로 통과한 뒤 목적지 도달 반경 안에 진입하면 1, 충돌·구역 이탈·시간 초과는 0. 각 경유점과 목적지의 도달 반경은 에피소드 시작 메타데이터에 포함됩니다.
F · 제어 세기	F_{best} / F_{agent} . F_{agent} 는 운항 동안 실제로 적용된 명령의 변화 를 누적인 값으로, 매 틱 $(\Delta v_{cmd} / V_{MAX})^2 + (\Delta \omega_{cmd} / W_{MAX})^2$ 를 더합니다(정지 상태에서 시작, $V_{MAX} \cdot W_{MAX}$ 는 명령 한계). 급한 속도·방향 변경이 잦을수록 커지고, 같은 세팅을 오래 유지하는 부드러운 운항일수록 작습니다. F_{best} 는 그 에피소드를 완주한 참가자 중 최솟값입니다. 자기 실행 기록의 $v_{cmd} \cdot w_{cmd}$ 로 그대로 계산할 수 있습니다.
C · 안전거리	C_{agent} / C_{best} . 안전거리(clearance)는 선체 표면과 장애물 표면 사이 거리(강둑 포함)를 매 틱 재고, 에피소드 동안 기록한 값 중 낮은 쪽 5%의 평균 을 씁니다. 틱별 값은 10 m에서 자릅니다. 위험했던 순간만 보는 통계이므로 넓은 물에서 멀리 도는 것은 값을 올리지 못합니다.
T · 시간 효율	T_{best} / T_{agent} . T_{agent} 는 실제 소요 시간, T_{best} 는 그 에피소드를 완주한 참가자 중 최단 시간입니다.

순간 최솟값은 한 프레임의 노이즈에 좌우됩니다. **C** 는 낮은 쪽 5%의 평균이므로 순간적인 스침과 지속적인 근접 운항을 구분합니다. 크게 우회해 안전거리만 벌리는 전략이 이득을 얻지 못하는 것은 이 백분위 때문입니다. 통계에 들어가는 것은 가장 가까웠던 순간들뿐이라 열린 물에서의 여유는 계산에 들어오지 않습니다. 안전거리를 넓히려 크게 돌거나 자주 방향을 바꾸면 시간 효율과 제어 세기에서 그만큼 감점되므로 3개 항목의 균형이 필요합니다.

시간 한도는 코스마다 다른 값이 아니라 **단일 공개 상수로**, 에피소드당 **6000 제어 틱**(600초)입니다. 한도에 도달하면 그 에피소드는 시간 초과로 종료되고, 위의 부분점수를 받습니다. 느린 운항 자체는 시간 효율 항목이 연속적으로 감점합니다.

최종 점수는 4가지 조건 점수의 **가중 평균**(1-1 20%, 1-2 20%, 1-3 30%, 1-4 30%)이며, 조건별 점수도 공개합니다. 모든 참가자가 동일한 비공개 에피소드 묶음으로 평가받으므로 조건별 난이도는 모든 참가자에게 같고, 조건별 강점과 약점은 조건별 점수로 드러납니다.

리더보드 읽는 법. 리더보드에는 규칙 설명이 없고 이 절이 기준입니다. 표의 한 행은 한 참가자이며, **참가자의 점수 제출물 가운데 가장 좋은 것이** 반영됩니다(가장 좋은 것이란 다른 참가자들의 반영 제출물을 상대로 최종 점수가 가장 높은 것, 같으면 먼저 낸 것). 열의 뜻: **final** 은 위의 최종 점수, **1-1 ~ 1-4** 는 관측 조건별 조건 점수(모두 $\times 100$), **completed** 는 완주한 에피소드 수 / 전체 에피소드 수, **counting** 은 반영된 제출물의 번호와 그 참가자의 점수 건수, **submitted** 는 그 제출물의 점수 시각입니다. **동점**은 반올림 전 전체 정밀도의 최종 점수로 먼저 가르고, 그래도 같으면 모든 에피소드에서 통과한 경유점 수의 합이 많은 참가자, 그래도 같으면 먼저 점수한 참가자가 앞섭니다. 완주가 하나도 없는 참가자는 모두 0점이라 이 순서가 실제로 쓰이는 곳은 표의 아래쪽입니다. 참가자 페이지에는 그 참가자의

모든 점수 제출물이 반영 여부와 함께, 그리고 반영 제출물의 조건별·에피소드별 점수, 실패 원인 분포, 제출물 결함(예외·잘못된 액션·시간 초과), 1-2 대비 1-4의 외란 성능 저하가 나옵니다. 대회 기간 중 리더보드는 그때까지 접수된 참가자들만을 상대로 같은 계산을 한 값이므로, 새 참가자가 들어오면 자기 점수가 움직일 수 있습니다.

진단 지표로는 실패 원인 분포(정적 장애물 충돌·이동 선박 충돌·구역 이탈·시간 초과)와 1-2·1-4 paired 스테이지의 외란 성능 저하율을 공개합니다. 둘 다 최종 점수에 반영하지 않습니다. 상황 유형별 완주율은 공개하지 않습니다 — 비공개 에피소드는 여러 상황이 이어진 복합 코스이고, 유형별로 쪼개면 표본이 너무 작아 진단으로 쓸 수 없기 때문입니다.

평가 조건	주요 평가 능력
1-1 Structured	perception을 제외한 planning·prediction·control
1-2 Top-view	영상 기반 상태 추출과 동적 객체 추적
1-3 LiDAR	부분 관측 누적과 기하 기반 회피
1-4 Top-view + drift	온라인 외란 추정과 적응·강건 제어

아직 확정되지 않은 값

비공개 에피소드 수 하나입니다. 위에 적힌 집계 절차, 항목 정의와 가중치, 조건 비중, 안전거리 통계, 시간 한도, 반영 제출물과 동점 처리는 확정된 내용이며 스타터 키트의 `RULES.md` 와 공개 채점기가 같은 값을 씁니다. 에피소드 수는 참가자 수가 정해지면 확정하고, 이후 조정이 불가피하면 변경 내용을 모든 참가자에게 사전 공지합니다.

제출 형식

공개·비공개 환경. 공개 환경(스타터 킷의 시뮬레이터·연습 코스)과 비공개 평가 환경은 같은 관측·액션 규격, 장애물 유형, 외란 유형을 사용합니다. 비공개 평가에는 새로운 지도, 출발·목표 지점, 장애물 배치와 공개 범위 안의 새로운 값·조합이 등장합니다. 에이전트는 채점기의 **별도 프로세스**에서 실행되며 관측과 액션만 주고받으므로 비공개 파라미터는 구조적으로 닿지 않습니다. 제출 디렉터리 바깥의 파일을 읽는 행위는 금지이며 적발 시 실격입니다. 에피소드가 바뀌면 `reset()` 이 호출되고, 이전 에피소드의 내부 상태를 다음 에피소드로 가져가서는 안 됩니다.

제출물은 디렉터리 하나입니다. 컨테이너 이미지가 아닙니다. 안에 들어가는 것은 다음 다섯 가지이며, 정확한 규격은 스타터 킷의 `docs/PACKAGING.md` 가 기준입니다.

submission.json

매니페스트. `format ( usvnav-submission/1 )`, `team`, `entry ( module:Class )`, `config`, `runtime ( usvnav-runtime/1 )` 다섯 키가 모두 필요합니다

agent.py

`Agent(config_path)` 로 생성되고 `reset(meta)` · `act(obs)` 를 갖는 클래스. 4가지 관측 조건을 하나의 구현이 처리합니다

config.json

내용은 자유. 절대 경로가 `Agent()` 에 전달됩니다

weights/

선택. 채점 환경에는 네트워크가 없으므로 모델이 읽는 것은 전부 디렉터리 안에 있어야 합니다

SUBMISSION.md

필수이지만 채점·검증하지 않습니다. 구조, 사용 데이터, 알려진 실패 조건을 적어 두면 문의 대응에 쓰입니다

✓ 디렉터리 크기 **200 MB 이하**(`__pycache__/` · `.git/` 제외). 사전학습 가중치와 외부 데이터의 사용은 제한하지 않으며 신고 의무도 없습니다.

✓ 채점 런타임은 Python 3.13, numpy, onnxruntime(GPU 빌드)뿐이며 버전은 스타터 킷의 `docs/RUNTIME.md` 에 고정돼 있습니다. 그 밖의 의존성은 디렉터리 안에 순수 파이썬으로 넣어야 합니다.

GPU는 채점 PC의 카드(NVIDIA GeForce RTX 5060, 8 GB)를 제출물 하나가 혼자 씁니다.

✓ 제출 전에 스타터 킷의 `usvnav validate-submission` 디렉터리 를 실행하십시오. 접수에서 적용하는 규칙을 같은 코드로 미리 검사하며, 여기서 거절되는 제출물은 채점되지 않습니다.

✓ **제출 방법.** 업로드에 쓰는 도구와 방법은 제출 접수가 열릴 때 이 페이지와 이메일로 안내합니다. 제출물은 위 형식의 디렉터리이며, 제출 도구는 업로드를 마친 뒤 같은 칸에 표식 파일 `<이름>.submit` (제출물의 체크섬 포함)을 놓습니다.

✓ **접수 규칙.** 올려 둔 파일은 그 자체로는 제출물이 아닙니다. 표식이 있고 그 체크섬이 올라온 파일과 일치할 때만 접수하며, 일치하면 바로 접수합니다. 표식 없는 업로드는 보지 않고 영수증도 없습니다. 같은 이름으로는 다시 올릴 수 없고(덮어쓰기 없음, 새 `--name`), 같은 바이트의 재제출은 `duplicate` 입니다. **마감**은 표식이 도착해 일치한 시점 기준이며, 마감 시각에 표식이 없거나 일치하지 않는 업로드는 제출물이 아닙니다.

`submission.json` 의 `team` 은 참가자 ID(참가자 폴더 이름)와 같아야 하며 다르면 접수가 거절됩니다. 참가자당 **24시간 이동 창에 채점 5건**까지이며, 초과분은 창이 비면 자동으로 채점됩니다. 모든 접수에는 영수증(`queued` / `scored` / `refused` / `duplicate`)이 `track1/.hangang/<이름>.json` 으로 발급됩니다.

✓ 리더보드에는 참가자의 **가장 좋은 접수 제출물**이 반영되고, 제출 이력은 참가자 페이지에 모두 남습니다.

규칙 전문은 스타터 킷의 `docs/RULES.md` , 제출물 규격은 `docs/PACKAGING.md` , 런타임은 `docs/RUNTIME.md` , 전달·접수 절차는 공통 규정 2절이 기준이며, 이 절은 그 요약입니다.